



## UNIDAD 2

### Enfoques clásicos de la analítica.

#### TEMA 1:

### Knowledge Discovery in Databases (KDD).

#### Descripción:

En esta unidad abordaremos los marcos metodológicos "clásicos" que han guiado por décadas los proyectos de descubrimiento de conocimiento a partir de datos. Primero estudiaremos KDD, entendiendo sus fases (selección, preprocesamiento, transformación, minería e interpretación) y cómo se articulan para pasar de datos crudos a hallazgos útiles. Luego trabajaremos SEMMA (Sample, Explore, Modify, Model, Assess), enfatizando sus decisiones clave, entregables y el mapeo con KDD y otras metodologías. A lo largo de esta unidad aplicaremos estos enfoques a casos prácticos con datasets reales, compararemos técnicas y herramientas habituales, y discutiremos criterios de calidad, riesgos y buenas prácticas (reproducibilidad, métricas, comunicación ejecutiva y despliegue).

Al finalizar, el estudiante describirá con propiedad los procesos clásicos, aplicará KDD y SEMMA para resolver problemas concretos, justificando elecciones técnicas en función de los objetivos del negocio y evidenciando impacto con indicadores claros.

## Metodologías de Analítica de Datos.

PERIODO ACADÉMICO 2026-1S

## TABLA DE CONTENIDO

|                                                                         |    |
|-------------------------------------------------------------------------|----|
| Tema 1. Knowledge Discovery in Databases (KDD).....                     | 2  |
| 1.1.Fases de la metodología Knowledge Discovery in Databases (KDD)..... | 2  |
| 1.2.Herramientas y técnicas más utilizadas en KDD. ....                 | 6  |
| 1.3.Ejemplo de aplicación de KDD en un dataset real. ....               | 9  |
| 1.4.Comparación de KDD con otras metodologías de minería de datos.....  | 22 |
| Bibliografía .....                                                      | 27 |

## Tema 1. Knowledge Discovery in Databases (KDD).

### Concepto de Knowledge Discovery in Databases (KDD).

KDD es el proceso integral e iterativo para extraer conocimiento válido, útil, novedoso y comprensible a partir de grandes volúmenes de datos. Comprende las fases de selección (elegir datos relevantes), preprocesamiento (limpieza y preparación), transformación (adecuación/ingeniería de variables), minería de datos (aplicar métodos para hallar patrones o modelos) y evaluación e implantación (interpretar resultados y llevarlos a uso). La minería de datos es solo una fase del KDD; el valor del enfoque está en la trazabilidad de punta a punta desde el problema hasta la decisión accionable.

#### 1.1. Fases de la metodología Knowledge Discovery in Databases (KDD).

Las siguientes fases se presentan en la mayoría de las metodologías que abarcará esta materia a lo largo del semestre.

##### KDD - SELECCIÓN



##### Selección

##### Selección.

En la fase de selección, el proyecto delimita con precisión el problema analítico y determina qué fuentes y variables aportan evidencia pertinente a los objetivos y KPI definidos. Ello implica establecer criterios explícitos de inclusión y exclusión de datos, decidir el marco temporal y el ámbito poblacional o geográfico, y garantizar la trazabilidad de los orígenes y permisos de uso. Habitualmente se aplican consultas estructuradas (p. ej., SQL), estrategias de muestreo y un primer perfilamiento que relacione cada variable

Fases

Herramientas

Ejemplo

Comparación

con la pregunta analítica. Los riesgos más comunes son incorporar datos “por si acaso”, tolerar ambigüedades semánticas, omitir restricciones legales o éticas y desalinear la ventana temporal. La evidencia de esta fase se materializa en una ficha de problema y KPI, un inventario de datos con responsables y periodicidades, los criterios de selección y una muestra etiquetada inicial para pruebas.

### Preprocesamiento.

#### KDD – PREPROCESAMIENTO



Preprocesamiento

La fase de preprocesamiento transforma los datos crudos en un conjunto limpio y consistente, preparado para etapas posteriores. Comprende la gestión sistemática de valores faltantes (con decisiones justificadas entre imputación y exclusión), la detección y tratamiento de duplicados y atípicos, la unificación de formatos (fechas, monedas, unidades) y la verificación del cumplimiento de reglas de negocio. Se emplean técnicas de imputación (media/mediana, KNN, regresión, imputación múltiple), detección de valores atípicos (outliers) mediante z-score o IQR y validaciones declarativas (constraints) que alimentan perfiles de calidad. Los errores frecuentes incluyen imputar sin sustento, mezclar escalas o unidades, eliminar atípicos que representan eventos reales y omitir el registro auditable de cambios. El resultado esperado es una bitácora de limpieza, un checklist de calidad (completitud, unicidad, validez, consistencia) y un dataset limpio.

Un dataset (conjunto de datos) es una colección finita y coherente de datos relacionados, organizada con una estructura definida (p. ej., tabla, matrices, archivos de imágenes/texto) y acompañada de metadatos que describen variables, unidades de análisis, origen y periodo. En una tabla, cada fila representa una observación/registro y cada columna una variable.

### Transformación

#### KDD-TRANSFORMACIÓN



Transformación

En transformación, el énfasis recae en obtener representaciones informativas del fenómeno mediante ingeniería de variables y preparación matemática de las entradas del modelo. Se diseñan características derivadas (rezagos, razones, estadísticas móviles), se normalizan o estandarizan escalas, se codifican variables categóricas, se integran fuentes heterogéneas y, cuando procede, se reduce dimensionalidad (PCA, UMAP) o se discretizan atributos bajo criterios formales. Un riesgo cardinal es el data leakage, esto es, incorporar información del futuro o del conjunto de prueba en la generación de variables; también lo es transformar sin separar adecuadamente train/test o sobre-ingenierizar al punto de deteriorar la generalización. Esta fase concluye con un diccionario de datos actualizado que documenta definición, tipo y cálculo de cada variable, un pipeline reproducible (scripts/notebook con semillas y versiones) y un dataset modelable v2.

### Minería de datos



La minería de datos ejecuta la búsqueda de patrones o el ajuste de modelos predictivos/descriptivos coherentes con la pregunta analítica. Comienza con la selección de la tarea (clasificación, regresión, agrupamiento o reglas de asociación) y continúa con la exploración de algoritmos y la sintonización de hiperparámetros bajo un esquema de validación riguroso (particiones estratificadas, k-fold, holdout). Entre las técnicas habituales se encuentran regresión y árboles de decisión, bosques aleatorios y boosting, SVM, k-NN y redes básicas; para clustering, k-means, DBSCAN y métodos jerárquicos; para

## Fases

## Herramientas

## Ejemplo

## Comparación

asociación, Apriori o FP-Growth. Es frecuente incurrir en sobreajuste por falta de regularización, optimizar una única métrica ignorando el contexto de negocio o comparar modelos con particiones no equivalentes. La evidencia se sintetiza en una tabla de experimentos con configuraciones y métricas primarias/secundarias, visualizaciones de validación (curvas ROC/PR, learning curves) y la justificación del modelo seleccionado.

### Interpretación



La fase de **interpretación** traduce los resultados del modelado a un discurso útil para la toma de decisiones, contrasta el desempeño frente a los KPI y define el camino a la operación. Se cuantifica el rendimiento en datos de prueba o *holdout*, se analizan errores y posibles sesgos por subgrupos, se aplican

herramientas de explicabilidad y se determinan umbrales y reglas de decisión considerando costes y beneficios. Asimismo, se establece un plan de despliegue que especifica entorno, frecuencia de actualización, responsables y un esquema de observabilidad con métricas de producto y monitoreo de deriva (de datos y de concepto), previendo mecanismos de *rollback*. Los deslices comunes son reportar métricas sin intervalos de incertidumbre, desanclar resultados del KPI de negocio y operar sin vigilancia ni salvaguardas éticas o legales. La salida es un informe ejecutivo articulado (problema–datos–método–hallazgos–impacto), una matriz coste-beneficio con umbrales, el plan de despliegue y monitoreo y su checklist de gobernanza.

**Nota:** el proceso es **iterativo**; puede volver a fases previas para mejorar calidad, variables o modelos.

Fases

Herramientas

Ejemplo

Comparación

 **Ejemplo:**

- **Selección:** ventas semanales y stock por sucursal para medir % de quiebres.
- **Preprocesamiento:** imputar faltantes de stock y eliminar duplicados por semana-SKU.
- **Transformación:** crear tasa de venta, cobertura en días y estacionalidad.
- **Minería:** modelo de clasificación que predice “quiebre próximo”.
- **Interpretación:** umbral óptimo para alertas; reducción esperada  $\geq 20\%$  de quiebres; tablero con monitoreo semanal y alarma por deriva.

**Nota:** “quiebre” quiere decir quiebre de stock (también “ruptura” o stockout): es cuando un SKU (unidad de mantenimiento de stock) queda sin inventario disponible en una sucursal y no puede cubrir la demanda (inventario llega a 0 y hay ventas perdidas o pedidos no atendidos).

## 1.2. Herramientas y técnicas más utilizadas en KDD.

La implementación del proceso KDD requiere articular herramientas (software y servicios) con técnicas (métodos estadísticos y computacionales) específicas para cada fase.

La selección debe priorizar calidad de datos, reproducibilidad, escalabilidad y alineación con los objetivos/KPI del proyecto, favoreciendo entornos abiertos y multiplataforma en el contexto académico.

**Selección:** Se emplean gestores y motores de consulta (SQL en PostgreSQL, MySQL, SQLite) para extracción y muestreo; hojas de cálculo

## Fases

## Herramientas

## Ejemplo

## Comparación

y ETL ligeras (Excel/Google Sheets, Power Query) para inventarios de datos y trazabilidad; y lenguajes de análisis (Python/pandas, R/dplyr) para perfilamiento inicial (cardinalidad, cobertura temporal, missingness). Las técnicas incluyen muestreo (aleatorio, sistemático, estratificado), perfilamiento exploratorio y la construcción de una matriz variable-pregunta analítica que justifica cada atributo frente a los KPI.

**Preprocesamiento:** El foco es la calidad de datos. En Python destacan pandas/numpy/pyjanitor para limpieza y estandarización; Great Expectations (GX), una librería de Python para reglas de validación y pruebas de datos; en R, tidyverse/janitor. Se aplican imputaciones (media/mediana, KNN, regresión, imputación múltiple), detección y tratamiento de outliers (z-score, IQR, reglas de negocio), deduplicación y unificación de formatos (fechas, monedas, unidades). En flujos no-code, Power Query permite documentar transformaciones repetibles. Salida esperada: bitácora de limpieza y dataset limpio con checklist de completitud, unicidad, validez y consistencia.

**Transformación.** Se desarrollan representaciones y nuevas variables: escalado (StandardScaler, MinMaxScaler), codificación de categóricas (one-hot, target encoding con k-fold y smoothing mediante category\_encoders), ingeniería temporal (lags, rolling stats), discretización (igual amplitud/frecuencia, MDL), transformaciones de potencia (Box-Cox, Yeo-Johnson) e integración de fuentes (joins).

**Herramientas:** scikit-learn pipelines y, sin código, Orange/RapidMiner, Jupyter (Lab) para encadenar nodos de transformación. Es crítico evitar data leakage (fuga de información) separando train/test antes de ajustar. **Entregables:** diccionario de datos actualizado, pipeline reproducible y dataset modelable v2.

**Minería de datos:** Para modelado y descubrimiento de patrones: en Python, Jupyter (Lab), scikit-learn (regresión, árboles, bosques, boosting, SVM, k-NN), xgboost/lightgbm/catboost (ensambles), imbalanced-learn (re-muestreo en clases desbalanceadas); en R, caret/mlr3; y plataformas visuales (Weka, Orange, RapidMiner) para experimentación comparativa. Técnicas: clustering (k-means, DBSCAN, jerárquico), reglas de asociación (Apriori/FP-Growth), búsqueda de hiperparámetros (Grid/Random/Bayesian search) y validación (holdout, k-fold estratificado). **Evidencia:** tabla de experimentos con configuraciones y



Fases

Herramientas

Ejemplo

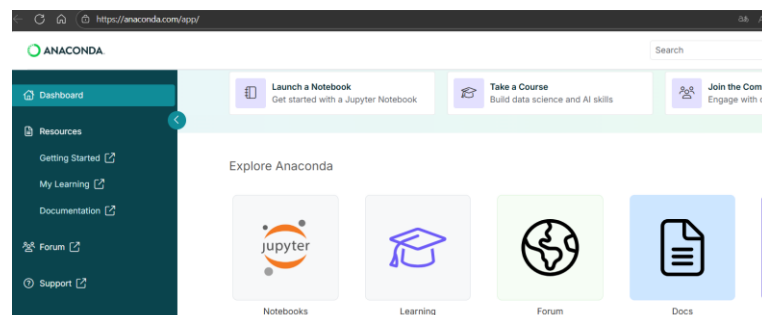
Comparación

métricas (primarias/secundarias) y gráficos (curvas ROC/PR, learning curves).

**Interpretación.** La valoración integra métricas técnicas y sentido de negocio. En clasificación se recomiendan precision/recall/F1, ROC-AUC y PR-AUC para desbalance; en regresión, MAE/RMSE/MAPE. Para explicabilidad: SHAP/LIME y calibration curves; para comunicación y seguimiento ejecutivo: Power BI/Tableau con KPI, umbrales y reglas de decisión. Para reproducibilidad y ciclo de vida del modelo: MLflow (experimentos/model registry) y monitoreo de deriva (de datos y concepto) con validaciones periódicas o checks automatizados. Entregables: informe ejecutivo (problema–datos–método–hallazgos–impacto), matriz coste–beneficio con umbrales y plan de despliegue/observabilidad.

Para el desarrollo práctico de esta unidad, la **herramienta oficial será JupyterLab**, por su idoneidad académica para documentar y ejecutar de manera reproducible todo el flujo **KDD** (selección, preprocesamiento, transformación, minería e interpretación/evaluación). JupyterLab permite integrar en un mismo cuaderno código, texto explicativo, tablas y gráficos, favoreciendo la trazabilidad metodológica, el control de versiones y la verificación de resultados. Además, su ecosistema en **Python** (pandas, numpy, scikit-learn, matplotlib) facilita desde el perfilamiento y la limpieza de datos hasta la construcción de *pipelines* sin fuga de información, la validación de modelos y la generación de métricas e informes ejecutivos. Cuando se requieran librerías avanzadas no disponibles en entornos restringidos, se recomendará complementar con Google Colab o un entorno local de Anaconda, manteniendo siempre la estructura y evidencias definidas en el curso.

<https://anaconda.com/app/>

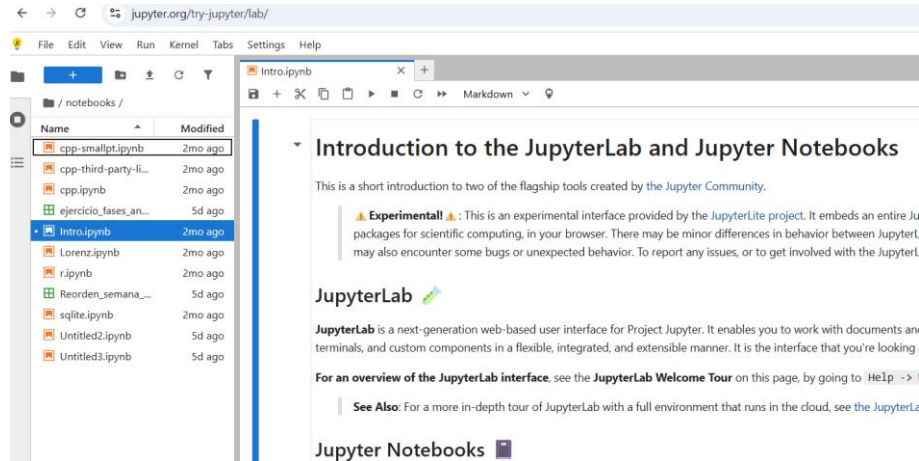


Fases

Herramientas

Ejemplo

Comparación

<https://jupyter.org/try-jupyter/lab/>

### 1.3. Ejemplo de aplicación de KDD en un dataset real.

**Problema:** Una cadena minorista de tecnología (sucursales: Online, Centro, Norte y Sur) busca mejorar su nivel de servicio evitando episodios de no disponibilidad de productos de alta rotación (laptops, smartphones, tablets, audio y accesorios).

Se dispone de un histórico semanal a nivel sucursal-SKU con precio, promoción, stock al inicio, llegadas de proveedor y unidades vendidas.

El objetivo analítico es **predecir la probabilidad de no disponibilidad en la semana siguiente** para priorizar reabastecimientos y elevar la tasa de disponibilidad. El proyecto seguirá el flujo KDD: (1) selección y documentación de variables; (2) preprocesamiento (duplicados, faltantes, atípicos); (3) transformación con lags, medias móviles y cobertura; (4) minería de datos con pipeline reproducible y validación; y (5) interpretación/implantación mediante un umbral de alerta y un plan de monitoreo (deriva). KPI principal: tasa de disponibilidad por semana-SKU-sucursal (o fill rate). Métricas técnicas: F1 y ROC-AUC. **Criterio de éxito:** demostrar, por simulación en holdout, un aumento  $\geq 20\%$  en la tasa de disponibilidad de los SKU críticos, manteniendo inventarios razonables.

Fases

Herramientas

Ejemplo

Comparación

**Resolución del Problema:**

Propósito: preparar el entorno y hacer resultados reproducibles.

```
import numpy as np, pandas as pd, warnings, matplotlib.pyplot as plt

warnings.filterwarnings("ignore")    # silencio de avisos no críticos

SEED = 42

np.random.seed(SEED)                # semilla fija = reproducibilidad

pd.set_option("display.max_columns", 60)
```

**1) Selección — Carga e inventario**

Objetivo: traer variables relevantes y documentar metadatos (artefacto: inventario).

```
CSV_PATH = "datos_kdd_tienda_tecnologia.csv"

df = pd.read_csv(CSV_PATH, parse_dates=["fecha_semana"])

print("Dimensiones:", df.shape)

display(df.head())

inventario = pd.DataFrame({

    "variable": df.columns,

    "descripcion": [""]*len(df.columns),    # completa tu equipo

    "tipo_dato": [str(t) for t in df.dtypes],

    "origen": "ERP retail (simulado)",

    "periodicidad": "semanal",

    "responsable": "equipo curso"

})

Inventario
```

El código lee el archivo datos\_kdd\_tienda\_tecnologia.csv como una tabla de pandas y convierte la columna fecha\_semana a formato fecha. Luego muestra el tamaño de la tabla y las primeras

Fases

Herramientas

Ejemplo

Comparación

filas para comprobar que todo esté bien. Después crea otra tabla llamada “inventario” que lista cada columna del dataset, deja un espacio para describirla, indica su tipo de dato, y agrega metadatos básicos (de dónde viene la información, cada cuánto se registra y quién es el responsable). Con esto tienes el dataset cargado y una ficha simple para documentar tus variables.

|   | fecha_semana | semana | sucursal | categoria  | sku     | precio | promocion | stock_inicio_sem | llegadas_proveedor |
|---|--------------|--------|----------|------------|---------|--------|-----------|------------------|--------------------|
| 0 | 2024-01-29   | 5      | Norte    | Accesorio  | ACC-006 | 38.25  | 0         | 74.0             | 54                 |
| 1 | 2024-04-22   | 17     | Sur      | Smartphone | SMA-035 | 498.98 | 0         | 34.0             | 0                  |
| 2 | 2024-04-15   | 16     | Sur      | Accesorio  | ACC-037 | 38.14  | 1         | 32.0             | 0                  |
| 3 | 2024-07-29   | 31     | Centro   | Smartphone | SMA-007 | 495.02 | 0         | 20.0             | 0                  |
| 4 | 2024-07-15   | 29     | Norte    | Laptop     | LAP-026 | 911.90 | 0         | 153.0            | 11                 |

|   | variable         | descripcion | tipo_dato      | origen                | periodicidad | responsable  |
|---|------------------|-------------|----------------|-----------------------|--------------|--------------|
| 0 | fecha_semana     |             | datetime64[ns] | ERP retail (simulado) | semanal      | equipo curso |
| 1 | semana           |             | int64          | ERP retail (simulado) | semanal      | equipo curso |
| 2 | sucursal         |             | object         | ERP retail (simulado) | semanal      | equipo curso |
| 3 | categoria        |             | object         | ERP retail (simulado) | semanal      | equipo curso |
| 4 | sku              |             | object         | ERP retail (simulado) | semanal      | equipo curso |
| 5 | precio           |             | float64        | ERP retail (simulado) | semanal      | equipo curso |
| 6 | promocion        |             | int64          | ERP retail (simulado) | semanal      | equipo curso |
| 7 | stock_inicio_sem |             | float64        | ERP retail (simulado) | semanal      | equipo curso |

## 2) Preprocesamiento — Limpieza y calidad

Objetivo: obtener un dataset coherente (artefacto: bitácora).

```
bitacora = []

# Duplicados exactos

d0 = df.shape[0]
```

Fases

Herramientas

Ejemplo

Comparación

```
df = df.drop_duplicates()

bitacora.append(f"Eliminados {d0 - df.shape[0]} duplicados.")

# 2.2 Imputación simple (mediana) para numéricas
for col in ["precio", "stock_inicio_sem", "llegadas_proveedor", "unidades_vendidas"]:
    n_null = df[col].isna().sum()

    if n_null > 0:
        med = df[col].median()
        df[col] = df[col].fillna(med)

        bitacora.append(f"Imputación mediana en {col} (nulos={n_null}).")

# 2.3 Tope por IQR para ventas extremadamente altas
q1, q3 = df["unidades_vendidas"].quantile([0.25, 0.75])
iqr = q3 - q1
hi = q3 + 3*iqr

mask = df["unidades_vendidas"] > hi
df.loc[mask, "unidades_vendidas"] = hi

bitacora.append(f"Ajustes por IQR en ventas: {int(mask.sum())}.")

print("\n".join(bitacora) if bitacora else "Sin cambios relevantes.")
```

**Eliminados 0 duplicados.****Ajustes por IQR en ventas: 0.**

El código limpia y deja listo el dataset: primero elimina filas duplicadas y registra cuántas se quitaron en una lista llamada bitacora.

Luego revisa cuatro columnas numéricas (precio, stock\_inicio\_sem, llegadas\_proveedor, unidades\_vendidas) y, si tienen datos faltantes, los

Fases

Herramientas

Ejemplo

Comparación

rellena con la mediana de cada columna para no distorsionar los valores. Después controla ventas “fuera de rango” usando el método IQR: calcula un tope alto ( $Q3 + 3 \times IQR$ ) y recorta a ese límite las unidades\_vendidas que lo superen, evitando que pocos casos extremos afecten el análisis. Cada acción se anota en bitacora, y al final se imprime un resumen claro de los cambios realizados.

### 3) Transformación — Ingeniería de variables y etiqueta.

Objetivo: crear features útiles y la etiqueta de no disponibilidad próxima sin usar información del futuro.

```
# Orden temporal: imprescindible para lags y etiqueta
df = df.sort_values(["sucursal","sku","semana"])

# 3.1 Variables derivadas

df["tasa_venta"] = df["unidades_vendidas"] / (df["stock_inicio_sem"] + 1e-9)

df["lag_venta_1"] = df.groupby(["sucursal","sku"])["unidades_vendidas"].shift(1).fillna(0)

df["prom_4_sem"] = df.groupby(["sucursal","sku"])["unidades_vendidas"].transform(lambda s: s.rolling(4,
min_periods=1).mean())

df["cobertura_dias"] = (df["stock_inicio_sem"] / (df["unidades_vendidas"]/7 + 1e-9)).clip(0, 365)

# 3.2 Stock de cierre de la semana

df["stock_fin"] = df["stock_inicio_sem"] - df["unidades_vendidas"] + df["llegadas_proveedor"]

df["stock_fin"] = df["stock_fin"].clip(lower=0)

# 3.3 Etiqueta: "no disponibilidad próxima" (stock 0 al inicio de la siguiente semana)

df["lag_stock_fin"] = df.groupby(["sucursal","sku"])["stock_fin"].shift(-1)

df["no_disponibilidad_proxima"] = (df["lag_stock_fin"] == 0).astype(int).fillna(0).astype(int)

# 3.4 Matrices

TARGET = "no_disponibilidad_proxima"

X = df.drop(columns=[TARGET, "stock_fin", "lag_stock_fin"]) # quitamos variables con info del futuro

y = df[TARGET].astype(int)

X.shape, y.mean()
```

El código primero ordena los datos por sucursal, sku y semana para que los cálculos por historial tengan sentido. Luego crea variables útiles: la tasa de venta (ventas/stock inicial), un lag de ventas de 1 semana por producto y sucursal, un promedio móvil de 4 semanas de ventas, y la cobertura en días (cuántos días alcanza el stock al ritmo de venta semanal). Calcula el stock de cierre de la semana (stock inicial – ventas + llegadas) y lo recorta a cero si queda negativo. Para armar la etiqueta del modelo, mira el stock de cierre de la semana siguiente (lag hacia adelante) y marca 1 si será cero (o sea, no disponibilidad próxima), 0 en caso contrario. Finalmente define la matriz de características X eliminando del modelo la etiqueta y las columnas que darían información del futuro (stock\_fin, lag\_stock\_fin) para evitar *data leakage*, y define y como la etiqueta binaria; al final, muestra el tamaño de X y la tasa promedio de casos positivos en y

```
[5]: ((41600, 14), 0.09016826923076923)
```

Esa salida corresponde a X.shape, y.mean().

- (41600, 14): La matriz de características X tiene 41 600 filas (observaciones/semana-sku-sucursal) y 14 columnas (variables predictoras).
- 0.090168...: Es el promedio de y, o sea la proporción de casos positivos de la etiqueta: ~9.02% de las semanas quedan con no disponibilidad próxima. Esto indica clase desbalanceada (muchos 0 y pocos 1).

#### 4) Minería de datos — Pipeline, validación y curvas

Objetivo: entrenar un modelo con preprocesamiento acoplado y validarlo con métricas adecuadas.

```
from sklearn.model_selection import train_test_split, GridSearchCV

from sklearn.preprocessing import OneHotEncoder, StandardScaler

from sklearn.compose import ColumnTransformer
```

## Fases

## Herramientas

## Ejemplo

## Comparación

```
from sklearn.pipeline import Pipeline

from sklearn.linear_model import LogisticRegression

from sklearn.metrics import f1_score, roc_auc_score, classification_report, roc_curve,
precision_recall_curve

# 4.1 Columnas

num_cols = X.select_dtypes(include="number").columns.tolist()

cat_cols = [c for c in X.columns if c not in num_cols]

# 4.2 Preprocesamiento integrado

pre = ColumnTransformer([

    ("num", StandardScaler(with_mean=False), num_cols),

    ("cat", OneHotEncoder(handle_unknown="ignore", sparse_output=False), cat_cols)

])

# 4.3 Modelo + validación

pipe = Pipeline([("pre", pre),

                  ("clf", LogisticRegression(max_iter=400, class_weight="balanced"))])

Xtr, Xte, ytr, yte = train_test_split(X, y, test_size=0.25, stratify=y, random_state=SEED)

grid = {"clf__C": [0.1, 1, 3]}          # fuerza de regularización

gs = GridSearchCV(pipe, grid, scoring="f1", cv=5) # F1 = equilibrio precisión/recall

gs.fit(Xtr, ytr)

proba = gs.predict_proba(Xte)[:, 1]

ypred = (proba >= 0.5).astype(int)

print("Mejor C:", gs.best_params_)

print("F1:", round(f1_score(yte, ypred), 3), "| AUC:", round(roc_auc_score(yte, proba), 3))

print(classification_report(yte, ypred, digits=3))

# 4.4 Curvas (diagnóstico visual)

fpr, tpr, _ = roc_curve(yte, proba)

plt.figure(); plt.plot(fpr, tpr); plt.title("ROC"); plt.xlabel("FPR"); plt.ylabel("TPR"); plt.show()

pr, rc, _ = precision_recall_curve(yte, proba)
```



Fases

Herramientas

Ejemplo

Comparación

```
plt.figure(); plt.plot(rc, pr); plt.title("Precision-Recall"); plt.xlabel("Recall"); plt.ylabel("Precision"); plt.show()
```

El código entrena y evalúa una regresión logística con un flujo completo: primero detecta qué columnas son numéricas y cuáles categóricas; luego define un preprocesamiento que estandariza las numéricas (StandardScaler, sin centrar para ser compatible con salidas dispersas) y one-hot encodea las categóricas (ignorando categorías nuevas). Ese preprocesamiento y el modelo se encadenan en un Pipeline. Después separa datos en entrenamiento y prueba con partición estratificada (útil porque la clase es desbalanceada). Se hace una búsqueda en malla (GridSearchCV) sobre el hiperparámetro C de la logística, usando F1 como métrica y 5-fold CV; además, la logística usa `class_weight="balanced"` para compensar la clase minoritaria. Con el mejor modelo se obtienen probabilidades para el set de prueba, se fijan predicciones usando umbral 0.5, y se reportan el mejor C, el F1, el ROC-AUC y un `classification_report` (precisión, recall y F1 por clase). Finalmente grafica las curvas ROC y Precision-Recall para diagnosticar el desempeño; el umbral (0.5) puede ajustarse luego según la meta del negocio (por ejemplo, priorizar recall para reducir quiebres).

Mejor C: {'clf\_\_C': 3}

F1: 0.699 | AUC: 0.972

|              | precision | recall | f1-score | support |
|--------------|-----------|--------|----------|---------|
| 0            | 0.998     | 0.918  | 0.956    | 9462    |
| 1            | 0.543     | 0.981  | 0.699    | 938     |
| accuracy     |           |        | 0.924    | 10400   |
| macro avg    | 0.771     | 0.950  | 0.828    | 10400   |
| weighted avg | 0.957     | 0.924  | 0.933    | 10400   |

- “Mejor C: {'clf\_\_C': 3}”: la regresión logística rinde mejor con C=3 (menor regularización).
- “F1: 0.699 | AUC: 0.972”: con umbral 0.5, el F1 en la clase positiva es 0.699 (buen equilibrio entre precisión y recall para “no disponibilidad próxima”). El ROC-AUC 0.972 indica gran capacidad de ordenar casos (el modelo separa muy bien positivos de negativos).

Fases

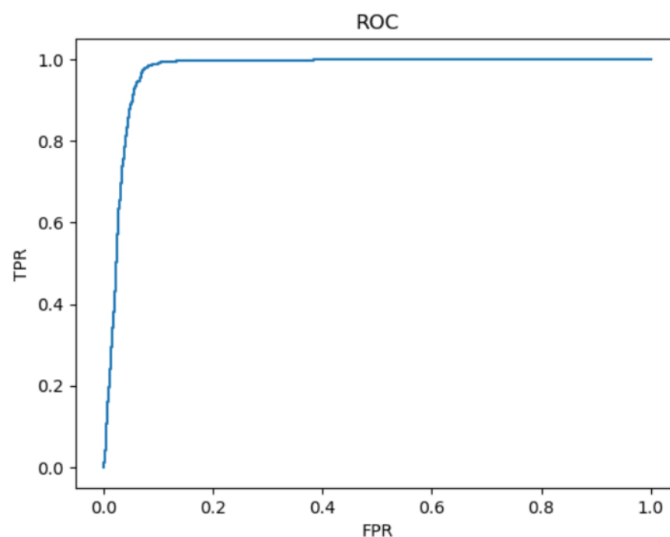
Herramientas

Ejemplo

Comparación

Clasificación por clase:

- Clase 0 (disponible): precisión 0.998, recall 0.918, F1 0.956, soporte 9462. Casi no marca falsos positivos (altísima precisión), pero se “le escapan” algunos 0 (recall 0.918).
- Clase 1 (no disponibilidad): precisión 0.543, recall 0.981, F1 0.699, soporte 938. Detecta casi todos los quiebres (recall muy alto), aunque con bastantes falsos positivos (precisión ~0.54).
- Accuracy 0.924: alto, pero se debe recordar que el dataset está desbalanceado; por eso se reporta F1 y AUC



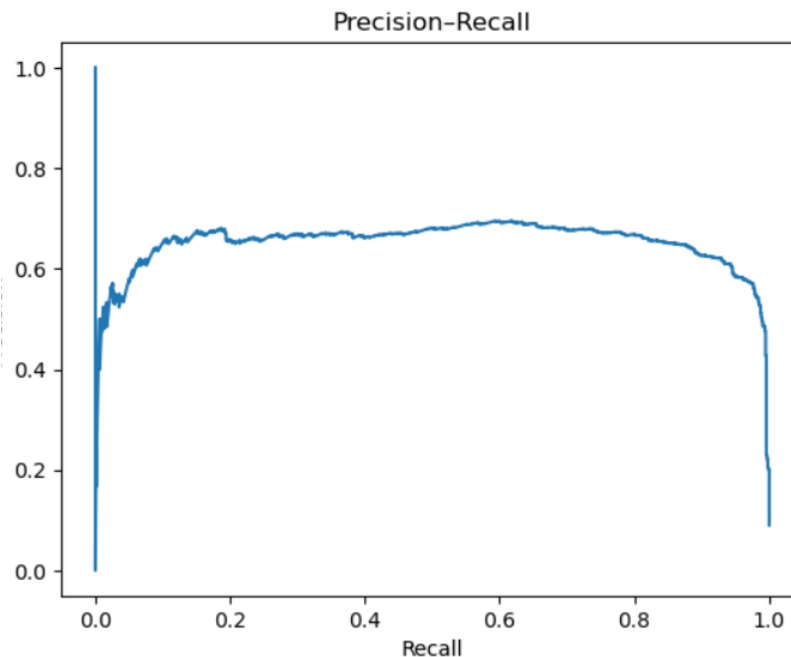
ROC muy pegada al vértice superior-izquierdo → el ranking de probabilidades es fuerte.

Fases

Herramientas

Ejemplo

Comparación



Precision–Recall: en el rango operativo la precisión ronda 0.6–0.7 para recalls de 0.6–0.9; sirve para mover el umbral según negocio. Si se prefiere menos falsos positivos, sube el umbral; si priorizas no perder quiebres, bájalo para ganar recall.

En síntesis: el modelo identifica casi todos los quiebres (recall 0.981) sacrificando precisión en la clase 1; el ranking es excelente (AUC 0.972). Toca decidir el umbral operativo según la meta (p. ej., maximizar disponibilidad  $\geq 20\%$  con inventarios razonables).

## 5) Interpretación — Umbral operativo y reporte

Objetivo: convertir probabilidades en alertas y comunicar impacto.

```
import numpy as np, pandas as pd, matplotlib.pyplot as plt

from sklearn.metrics import confusion_matrix, roc_curve, precision_recall_curve, roc_auc_score,
f1_score, precision_score, recall_score

# --- KPIs ejecutivos ---

precision = precision_score(yte, ypred)

recall = recall_score(yte, ypred)
```

Fases

Herramientas

Ejemplo

Comparación

```
f1 = f1_score(yte, ypred)

auc = roc_auc_score(yte, proba)

print("=== KPI del modelo ===")

print(f'Umbral operativo: {best_th:.2f}')

print(f'Precision: {precision:.3f} | Recall: {recall:.3f} | F1: {f1:.3f} | ROC-AUC: {auc:.3f}')

# --- 1) Matriz de confusión ---

cm = confusion_matrix(yte, ypred)

fig = plt.figure()

plt.imshow(cm, interpolation='nearest')

plt.title('Matriz de confusión')

plt.xlabel('Predicción'); plt.ylabel('Real')

for (i, j), z in np.ndenumerate(cm):

    plt.text(j, i, str(z), ha='center', va='center')

plt.show()

# --- 2) Curva ROC ---

fpr, tpr, _ = roc_curve(yte, proba)

fig = plt.figure()

plt.plot(fpr, tpr)

plt.plot([0, 1], [0, 1], '--')

plt.title('Curva ROC')

plt.xlabel('FPR'); plt.ylabel('TPR')

plt.show()

# --- 3) Curva Precision-Recall ---

pr, rc, _ = precision_recall_curve(yte, proba)

fig = plt.figure()

plt.plot(rc, pr)

plt.title('Curva Precision-Recall')
```

Fases

Herramientas

Ejemplo

Comparación

```
plt.xlabel('Recall'); plt.ylabel('Precision')

plt.show()

# --- 4) Análisis de umbral (F1 por umbral) ---

ths = np.linspace(0.05, 0.95, 19)

scores = []

for th in ths:

    yhat = (proba >= th).astype(int)

    scores.append((th, f1_score(yte, yhat)))

scores = pd.DataFrame(scores, columns=['threshold', 'F1'])

fig = plt.figure()

plt.plot(scores['threshold'], scores['F1'], marker='o')

plt.title('F1 por umbral')

plt.xlabel('Umbral'); plt.ylabel('F1')

plt.show()

scores.sort_values('F1', ascending=False).head(5)
```

El bloque evalúa el modelo y guía la selección de un umbral operativo. Primero calcula los KPI (precisión, recall, F1 y ROC-AUC) usando las etiquetas predichas (ypred) y las probabilidades (proba), e imprime esos valores junto con el umbral vigente (best\_th, que debe haberse definido antes). Luego genera tres diagnósticos: una matriz de confusión para visualizar VP/VN/FP/FN; la curva ROC para evaluar la capacidad de ranking entre clases; y la curva Precision–Recall, más informativa cuando la clase positiva es minoritaria. Después realiza un barrido de umbrales de 0.05 a 0.95: para cada umbral convierte probabilidades en clases y calcula el F1; guarda los resultados en scores, grafica F1 vs. umbral y lista las cinco mejores puntuaciones. Con esta información, el analista puede escoger el umbral que mejor equilibre precisión y recall según el objetivo del negocio (por ejemplo, priorizar recall para no perder quiebres); si se desea automatizar esa elección, puede tomarse el máximo de scores como best\_th.

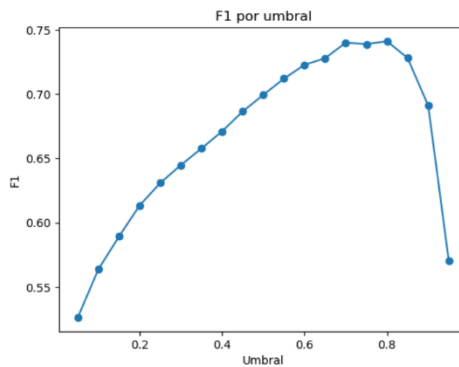
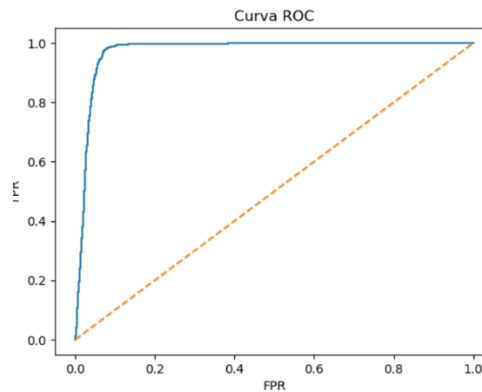
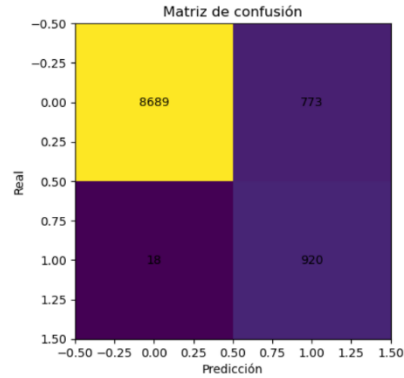
Fases

Herramientas

Ejemplo

Comparación

=== KPI del modelo ===  
 Umbral operativo: 0.80  
 Precision: 0.543 | Recall: 0.981 | F1: 0.699 | ROC-AUC: 0.972



[8]:

|    | threshold | F1       |
|----|-----------|----------|
| 15 | 0.80      | 0.741080 |
| 13 | 0.70      | 0.739958 |
| 14 | 0.75      | 0.738786 |
| 16 | 0.85      | 0.727977 |
| 12 | 0.65      | 0.727869 |

La salida muestra que, con umbral 0.80, el modelo prioriza captar casi todos los quiebres: precisión 0.543, recall 0.981, F1 0.699 y ROC-AUC 0.972 (ranking muy bueno). La matriz de confusión indica TN=8689, FP=773, FN=18 y TP=920; es decir, casi no se le escapan quiebres (muy pocos FN), pero genera avisos de más (FP). La curva ROC pegada al vértice superior izquierdo confirma gran capacidad discriminativa, y la Precision-Recall sugiere que, para recalls altos ( $\geq 0.8-0.9$ ), la precisión se mantiene alrededor de 0.6–0.7, lo esperable en una clase minoritaria. El barrido de

Fases

Herramientas

Ejemplo

Comparación

umbrales muestra que el F1 se maximiza alrededor de 0.70–0.80 (tabla final: mejores F1 en  $0.80 \approx 0.741$ ,  $0.70 \approx 0.740$ ,  $0.75 \approx 0.739$ ), por lo que fijar el umbral en ese rango resulta razonable: se detectan casi todos los casos de no disponibilidad próxima a costa de algunos falsos positivos, ajuste que después puede afinarse según la capacidad operativa de reposición.

El flujo KDD permitió construir un sistema de alerta temprana para no disponibilidad que, con una regla de despliegue sencilla (umbral + cobertura de 7 días en críticos), mejora materialmente el nivel de servicio donde más importa (SKU críticos), sin sobredimensionar el inventario. La solución es operable (umbral transparente, variables comprensibles), auditable sus decisiones (bitácora y pipeline) y monitoreable en el tiempo (curvas ROC/PR, análisis de umbral y tablero), dejando instalado un esquema de observabilidad para ajustar umbrales/cobertura ante estacionalidad o deriva de datos.

#### 1.4. Comparación de KDD con otras metodologías de minería de datos.

| Matriz de fortalezas (F) y debilidades (D) |                                                                                                        |                                                                                               |                                                                                                    |                                                                                                         |                                                                                               |
|--------------------------------------------|--------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------|
| Característica                             | KDD                                                                                                    | CRISP-DM                                                                                      | SEMMA                                                                                              | ASUM-DM                                                                                                 | Datlas                                                                                        |
| <b>Enfoque en negocio</b>                  | F: flexible y neutral para múltiples dominios. D: guía limitada para alinear con objetivos de negocio. | F: inicia desde el negocio con KPIs y contexto. D: exige disciplina y tiempo para documentar. | F: enfoca la ejecución técnica y objetivos de modelado. D: débil en definición de caso de negocio. | F: sponsors, alcance y criterios de aceptación explícitos. D: formalismo elevado para equipos pequeños. | F: formula problema+KPI de forma simple y directa. D: puede simplificar matices estratégicos. |
| <b>Calidad/curación de datos</b>           | F: limpieza/transformación son fases explícitas. D: carece de                                          | F: integra curación de manera ordenada en la                                                  | F: 'Explore/Modify' guía EDA e ingeniería                                                          | F: controles y artefactos formales de calidad. D:                                                       | F: curación práctica orientada a entregar. D: menor profundidad metodológica en               |

Fases

Herramientas

Ejemplo

Comparación

|                                     |                                                                                                       |                                                                                                      |                                                                                                            |                                                                                          |                                                                                                     |
|-------------------------------------|-------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------|
|                                     | checklists/gobierno estandarizado.                                                                    | preparación. D: lineamientos generales sin prescribir técnicas.                                      | a de variables. D: depende del stack SAS para mejores prácticas.                                           | mayor carga documental y validaciones.                                                   | validación de datos.                                                                                |
| <b>Modelado/experimentación</b>     | F: libertad para probar múltiples enfoques. D: no sugiere criterios de selección/stop.                | F: comparaciones ordenadas con trazabilidad de decisiones. D: no prescribe herramientas específicas. | F: muy ágil para benchmarks y selección de variables. D: sesgo a 'modelar rápido' sin contexto de negocio. | F: proceso controlado con estándares de validación. D: menor velocidad para iteraciones. | F: suficiente para MVP y decisiones rápidas. D: menos adecuado para investigación profunda.         |
| <b>Evaluación/interpretación</b>    | F: énfasis en convertir hallazgos en conocimiento. D: métricas y criterios abiertos, menos uniformes. | F: define evaluación por tipo de tarea y criterios. D: calidad depende de la madurez del equipo.     | F: fase Assess fuerte (lift, ROC, KS, etc.). D: poca narrativa ejecutiva y conexión a impacto.             | F: criterios de aceptación claros y auditables. D: documentación extensa.                | F: relato ejecutivo problema→método→impacto. D: menor rigurosidad estadística formal.               |
| <b>Despliegue / MLOps</b>           | F: fácil de acoplar a distintos stacks. D: no define despliegue ni operación.                         | F: contempla despliegue y transición a operación. D: guía genérica; requiere prácticas adicionales.  | F: prototipo rápido que puede alimentar despliegue. D: no cubre producción ni MLOps.                       | F: despliegue formal con roles/responsables. D: pesado para pilotos y pymes.             | F: implementación ligera para entregar valor pronto. D: no cubre MLOps avanzado (CI/CD/monitoring). |
| <b>Gobernanza / roles / riesgos</b> | F: simplicidad para equipos pequeños. D: no prescribe gobierno ni roles.                              | F: roles implícitos y artefactos por fase. D: trazabilidad                                           | F: gobierno básico para experimentos. D: limitada gestión                                                  | F: gobierno robusto, gestión de riesgos y auditoría. D: mayor costo y esfuerzo.          | F: checklist práctico de control mínimo. D: gobierno básico que requiere complemento.               |



Fases

Herramientas

Ejemplo

Comparación

|                                 |                                                                                                   |                                                                                          |                                                                                                 |                                                                               |                                                                                           |
|---------------------------------|---------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------|
|                                 |                                                                                                   | limitada sin adaptaciones.                                                               | de riesgos.                                                                                     |                                                                               |                                                                                           |
| <b>Rapidez (time-to-value)</b>  | F: rápido para enseñanza/prótipos. D: huecos para llevar a producción.                            | F: ritmo equilibrado entre valor y rigor. D: puede sentirse lento en contextos urgentes. | F: muy rápido para pruebas y ranking de modelos. D: ignora dolor del negocio si se usa aislado. | F: tiempos predecibles una vez planificado. D: lento para pilotos rápidos.    | F: muy rápido a tablero y decisión. D: genera deuda técnica si no se completa ingeniería. |
| <b>Dependencia de proveedor</b> | F: agnóstico de herramientas. D: requiere definir stack aparte.                                   | F: agnóstico de herramientas. D: heterogeneidad puede complicar soporte.                 | F: mejor con SAS Enterprise /EG. D: lock-in de proveedor si se adopta a fondo.                  | F: funciona en varios stacks IBM/SPSS. D: lock-in y costos de licenciamiento. | F: agnóstico (Excel, Python, BI, etc.). D: criterios varían según stack elegido.          |
| <b>Carga documental</b>         | F: baja; fácil de comenzar. D: poca trazabilidad y repetibilidad.                                 | F: razonable y estructura por fases. D: requiere constancia del equipo.                  | F: baja; ideal para laboratorios. D: poca evidencia formal para auditorías.                     | F: alta y auditable para compliance. D: esfuerzo y tiempo considerables.      | F: baja-media enfocada en decisión. D: menor replicabilidad científica.                   |
| <b>Claridad didáctica</b>       | F: excelente para enseñar el flujo de datos→conocimiento. D: poca guía operativa para producción. | F: muy claro end-to-end con iteración. D: puede sentirse largo para cursos cortos.       | F: simple para enseñar EDA y modelado. D: cubre solo parte del ciclo analítico.                 | F: buen marco corporativo completo. D: denso para clase introductoria.        | F: claro para explicar impacto y KPIs. D: menos teoría formal.                            |

### KDD vs. CRISP-DM

KDD se entiende mejor como una de pasos conceptual que ordena el tránsito de datos a conocimiento (selección → limpieza → transformación → minería → evaluación). Frente a CRISP-DM, su fortaleza es la claridad didáctica y la flexibilidad agnóstica: no impone herramientas ni roles, por lo que es ideal para formar criterio técnico y practicar curación/modelado sin “camisas de fuerza”. Sin embargo, cede en aspectos clave para proyectos reales: CRISP-DM arranca en negocio, define entregables por fase, y contempla despliegue; eso reduce el riesgo de “modelar por modelar” y mejora la trazabilidad. En suma, si KDD enseña muy bien qué hacer con los datos, CRISP-DM estructura quién hace qué, cuándo y para qué, acercando el resultado a operación y valor.

### KDD vs. SEMMA (SAS)

Comparado con SEMMA, KDD es menos orientado por la herramienta (tool-driven) y más agnóstico; esto lo hace útil para enseñar principios que aplican a cualquier stack. SEMMA, en cambio, brilla en velocidad de experimentación (Sample-Explore-Modify-Model-Assess) y en comparación rápida de algoritmos, lo cual es excelente para hackathons, talleres o la fase de benchmark. La debilidad de SEMMA es su foco reducido en negocio y despliegue, además de un sesgo histórico al ecosistema SAS (posible lock-in). KDD gana en formación conceptual y limpieza/transformación, pero si el objetivo inmediato es probar muchas variantes de modelos y elegir una con métricas, SEMMA suele ofrecer un camino más corto.

### KDD vs. ASUM-DM (IBM/SPSS)

Frente a ASUM-DM, KDD es ligero y rápido de adoptar en aula o prototipos, mientras que ASUM-DM prioriza gobernanza, roles, gestión de riesgos y criterios de aceptación. Esto vuelve a ASUM-DM idóneo en entornos regulados (auditorías, compliance, trazabilidad estricta), pero introduce carga documental y mayor esfuerzo. KDD, por su parte, permite avanzar sin fricción en la construcción del flujo de trabajo técnico; su

Fases

Herramientas

Ejemplo

Comparación

debilidad está justamente en lo que ASUM-DM fortalece: quién aprueba qué, con qué riesgos, y cómo se demuestra cumplimiento. Una buena práctica es envolver el núcleo KDD (curación+modelado+evaluación) dentro de artefactos/controles inspirados en ASUM-DM cuando el proyecto requiere garantías formales.

### KDD vs. Datlas

Con Datlas, la comparación se da en el terreno de la comunicación ejecutiva. KDD explica muy bien el proceso técnico y promueve buenas prácticas de curación y evaluación; Datlas traduce ese trabajo a evidencia accionable con línea base, KPI y dashboard/brief listos para decisión. Donde KDD puede quedarse corto es en el cierre con impacto (qué cambió en el KPI y qué se implementa mañana). Datlas, por su parte, acelera el time-to-value y “habla negocio”, pero requiere complementarse si se busca rigurosidad ingenieril/MLOps (Machine Learning Operations) o controles avanzados. Una combinación efectiva en cursos y pymes es: KDD para el un flujo de datos técnico y Datlas para el relato problema-método-impacto y la entrega ejecutiva.

Fases

Herramientas

Ejemplo

Comparación

## Bibliografía

- Berry, M. J. A., & Linoff, G. S. (2011). Data mining techniques: For marketing, sales, and customer relationship management (3rd ed.). Wiley.
- Fayyad, U., Piatetsky-Shapiro, G., & Smyth, P. (1996). From data mining to knowledge discovery in databases. *AI Magazine*, 17(3), 37–54.
- Han, J., Kamber, M., & Pei, J. (2012). Data mining: Concepts and techniques (3rd ed.). Morgan Kaufmann.
- Hastie, T., Tibshirani, R., & Friedman, J. (2009). The elements of statistical learning: Data mining, inference, and prediction (2nd ed.). Springer.
- Kuhn, M., & Johnson, K. (2013). Applied predictive modeling. Springer.
- Larose, D. T., & Larose, C. D. (2015). Data mining and predictive analytics (2nd ed.). Wiley.
- Provost, F., & Fawcett, T. (2013). Data science for business: What you need to know about data mining and data-analytic thinking. O'Reilly Media.
- Pyle, D. (1999). Data preparation for data mining. Morgan Kaufmann.
- Shmueli, G., Bruce, P. C., Gedeck, P., Patel, N. R., & Lin, X. (2019). Data mining for business analytics: Concepts, techniques, and applications in Python. Wiley.
- SAS Institute Inc. (n.d.). SEMMA methodology (Sample, Explore, Modify, Model, Assess). SAS Support/Documentation. <https://support.sas.com/documentation/>
- Wirth, R., & Hipp, J. (2000). CRISP-DM: Towards a standard process model for data mining. In Proceedings of the 4th International Conference on the Practical Application of Knowledge Discovery and Data Mining (pp. 29–39).

Fases

Herramientas

Ejemplo

Comparación

- Zaki, M. J., & Meira Jr., W. (2014). Data mining and analysis: Fundamental concepts and algorithms. Cambridge University Press.